الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري

Arab Academy for Science, Technology & Maritime Transport

Lecture 4

Addressing Modes

Addressing Modes in Microprocessors

Overview:

When the 8088 microprocessor executes an instruction, it performs a specific function on data, referred to as operands. These operands may either:

- **Be directly part of the instruction itself,**
- **Reside in one of the internal registers, or**
- **Be stored in memory.**

To facilitate access to these operands, the 8088 uses various addressing modes, which are methods for specifying the location or value of the operand.

Types of Addressing Modes

1. Register Operand Addressing Mode

In the Register Operand Addressing Mode, operands are located in the internal registers of the microprocessor. This mode is highly efficient as it does not require additional memory access, making it faster than memory-based operations.

- Example:

```
MOV AX, BX
```

This instruction transfers the contents of the BX register into the AX register. No memory access is required, as both AX and BX are registers.

2. Immediate Operand Addressing Mode

In Immediate Addressing Mode, the operand is a fixed, constant value provided directly within the instruction itself. This is typically used to load specific values into registers without needing to access memory.

- Example:

```
MOV AL, 15h
```

This command moves the hexadecimal value 15h directly into the AL register. Here, 15h is the operand and is immediately available as part of the instruction.

3. Memory Addressing Modes

In Memory Addressing Modes, the operand is located in memory, requiring the calculation of a Physical Address (PA) to access it. The physical address is determined based on various factors, depending on the specific memory addressing mode used (such as direct, indirect, based, or indexed).

• Physical Address (PA) Calculation:

The PA represents the exact memory location where the operand resides. For example, it may involve adding base addresses, offsets, or register values. This calculation is crucial as it directs the microprocessor to the correct memory location for accessing the operand.

$$PA = \begin{bmatrix} CS \\ SS \\ DS \\ ES \end{bmatrix} : \begin{bmatrix} BX \\ BP \end{bmatrix} + \begin{bmatrix} SI \\ DI \end{bmatrix} + \begin{bmatrix} 8 \text{ bit displacement} \\ 16 \text{ bit displacement} \end{bmatrix}$$

Each addressing mode in memory may require a unique approach to determining the PA based on the operand's location and the method of access.

1- Register Operand Addressing Mode

In this mode, operands are located in the internal registers of the microprocessor. For example:

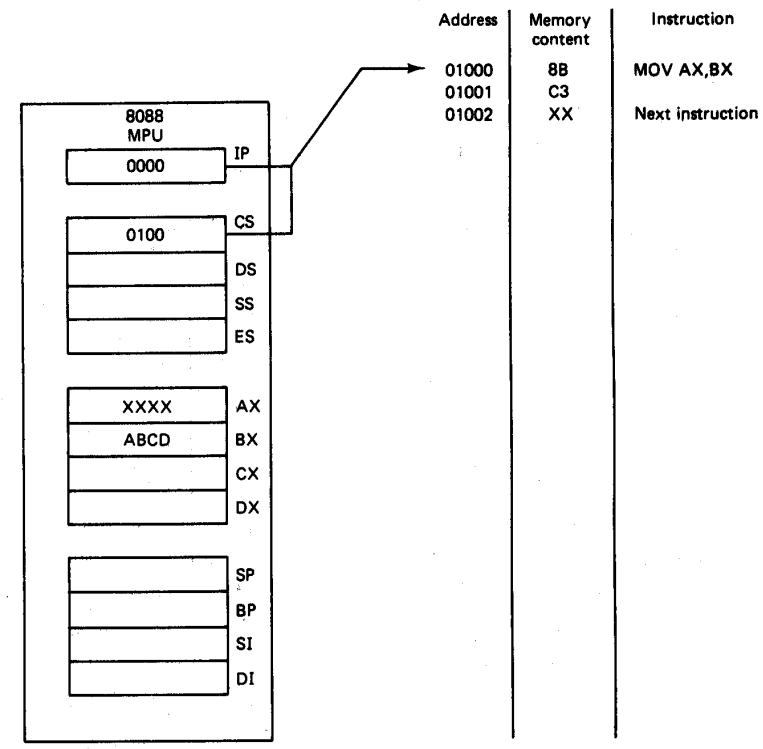
MOV AX, BX

This instruction moves the contents of register BX into register AX.

The machine code for the instruction `MOV AX, BX` is `C3 8B`.

This indicates that the instruction `MOV AX, BX`, which transfers the contents of register `BX` into `AX`, corresponds to the hexadecimal machine code `C3 8B` in assembly language.

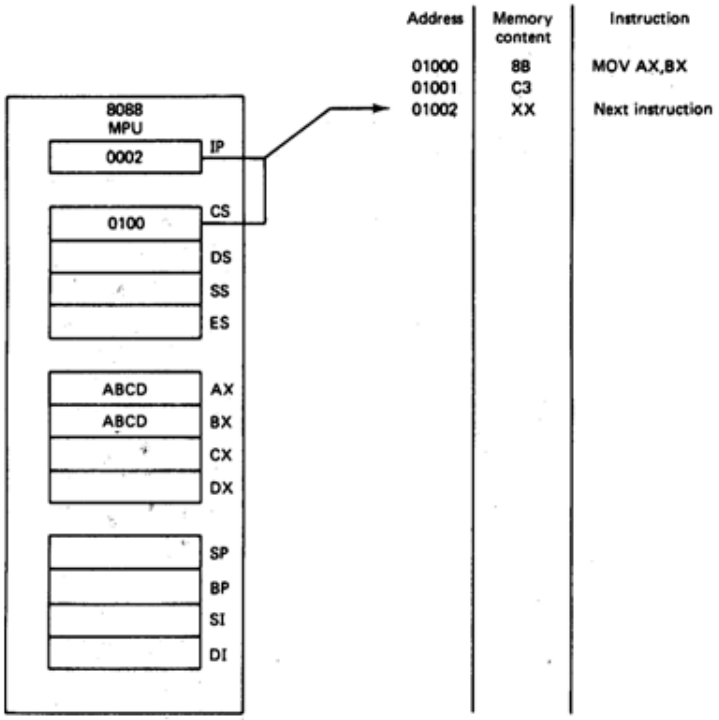
Before execution



(a)

Register Operand Addressing Mode (Cont.)

- After execution



(b)

This method is efficient as it involves no additional memory access. Register-based operations are typically faster compared to memory-based operations.

2. Immediate Operand Addressing Mode

Opcode	Immediate operand

In the Immediate Addressing Mode, the source operand is a constant value provided as part of the instruction. This is often used to load specific values into registers. For instance:

MOV AL, 15h;

This moves the hexadecimal value 15 directly into the AL register.

MOV AX, 2550h;

This instruction places the value 2550h into the AX register.

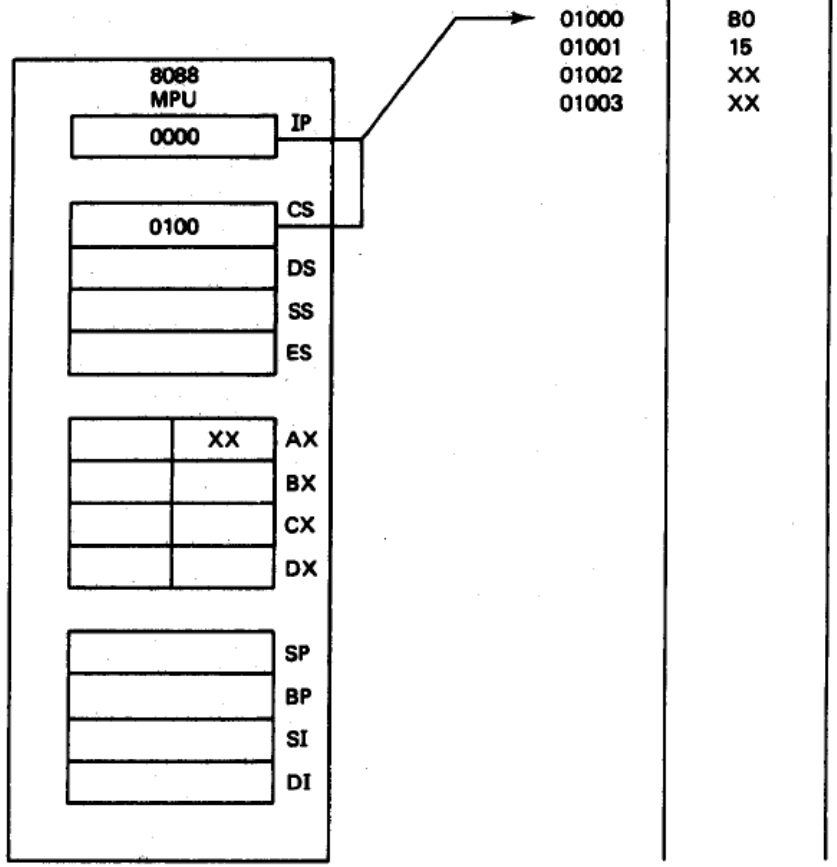
Using this mode simplifies loading constant values into registers without needing memory access.

Example

MOV AL, 15H

$15B0_{16}$

Before execution

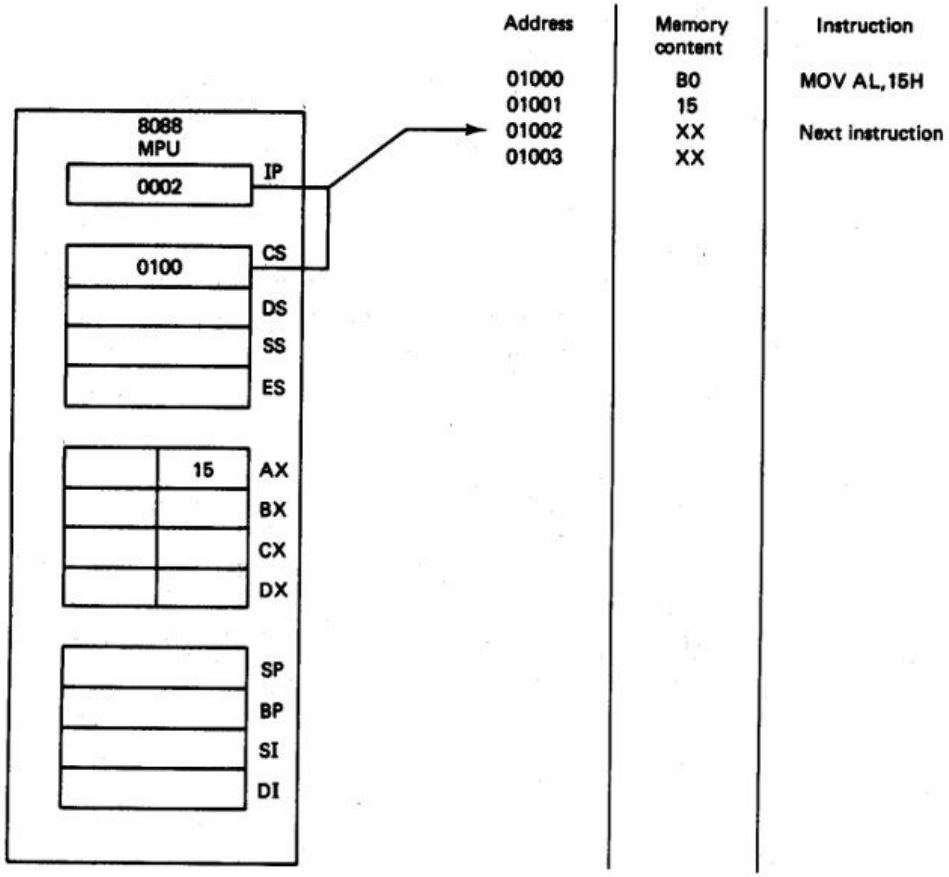


Address	Memory content	Instruction
01000	B0	MOV AL,15H
01001	15	
01002	XX	Next instruction
01003	XX	

(a)

Solution

After execution



(b)

3. Memory Addressing Mode

Memory Addressing involves calculating the Physical Address (PA) to access the operand stored in memory.
There are several variations of memory addressing modes:

$$PA = \begin{bmatrix} CS \\ SS \\ DS \\ ES \end{bmatrix} : \begin{bmatrix} BX \\ BP \end{bmatrix} + \begin{bmatrix} SI \\ DI \end{bmatrix} + \begin{bmatrix} \text{8 bit displacement} \\ \text{16 bit displacement} \end{bmatrix}$$

1. **Direct Addressing Mode**

   In this mode, the address of the operand is explicitly specified within the instruction.

   - *Example*: `MOV CX, [1234h]` – Moves the contents of memory location `1234h` into the `CX` register.

2. **Register Indirect Addressing Mode**

   Here, the memory address is indirectly specified by a register, often used for dynamic data access.

   - *Example*: `MOV AX, [SI]` – Moves data from the address in the `SI` (Source Index) register to `AX`.

3. **Based Addressing Mode**

   This mode calculates the address by adding a base register to a constant offset.

   - *Example*: `MOV CX, [BX] + 10` – Moves the data from the address `BX + 10` into `CX`.

4. **Indexed Addressing Mode**

   An index register, along with an offset, determines the memory address in this mode.

   - *Example*: `MOV DX, [SI] + 5` – Moves data from `SI + 5` to the `DX` register.

5. **Based-Indexed Addressing Mode**

   This combines both a base register and an index register, with an optional displacement to calculate the address.

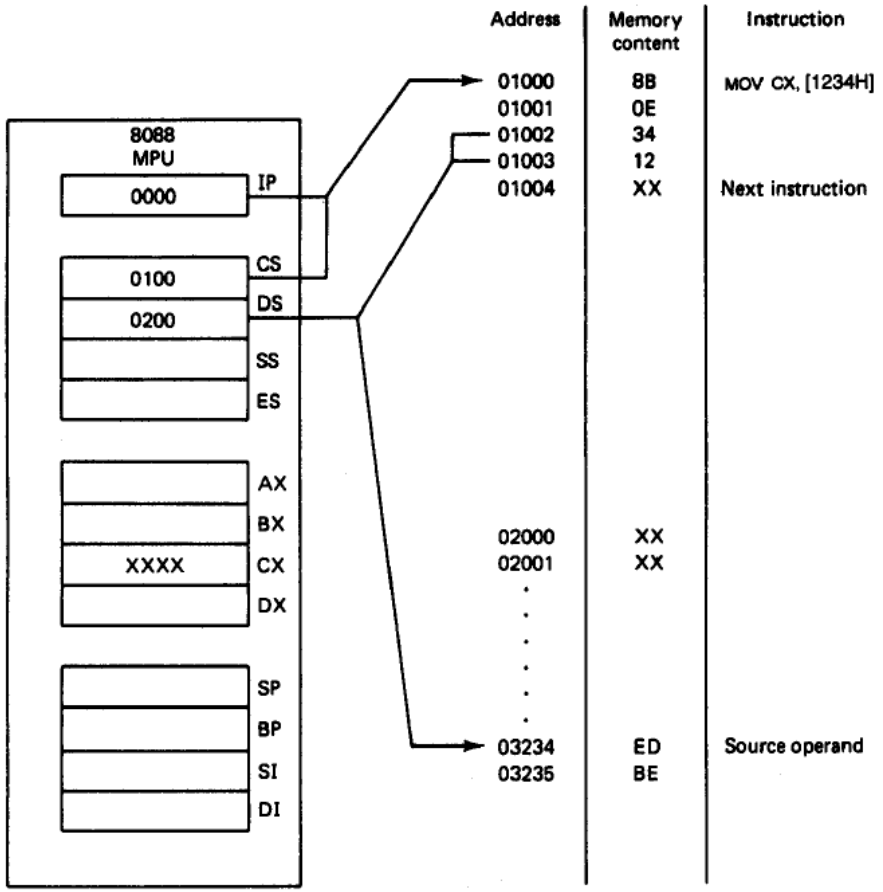
   - *Example*: `MOV CX, [BX][DI] + 8h` – Moves data from the address calculated by adding `BX`, `DI`, and `8h` into `CX`.

Direct Addressing Mode

MOV CX, [1234h]
$12340E8B_{16}$

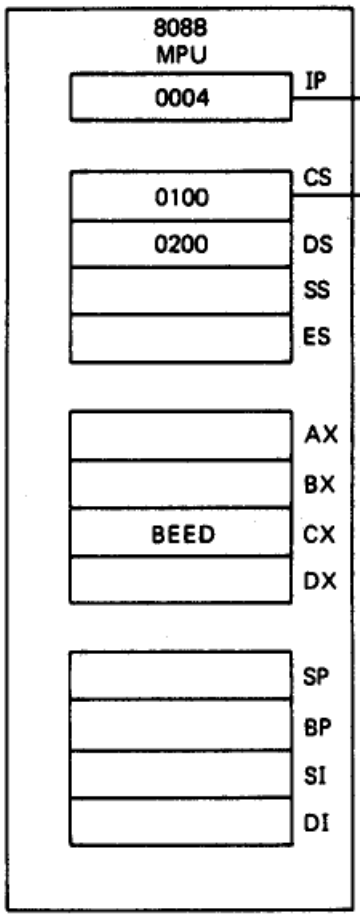
MOV CX, [1234h]: This instruction moves the value stored at the memory location `1234h` into the `CX` register. The PA is calculated directly using the specified address.

Before execution



(a)

After Execution



Address	Memory content	Instruction
01000	8B	MOV CX, [1234H]
01001	0E	
01002	34	
01003	12	
01004	XX	Next instruction
02000	XX	
02001	XX	
.		
.		
.		
.		
.		
03234	ED	
03235	BE	

(b)

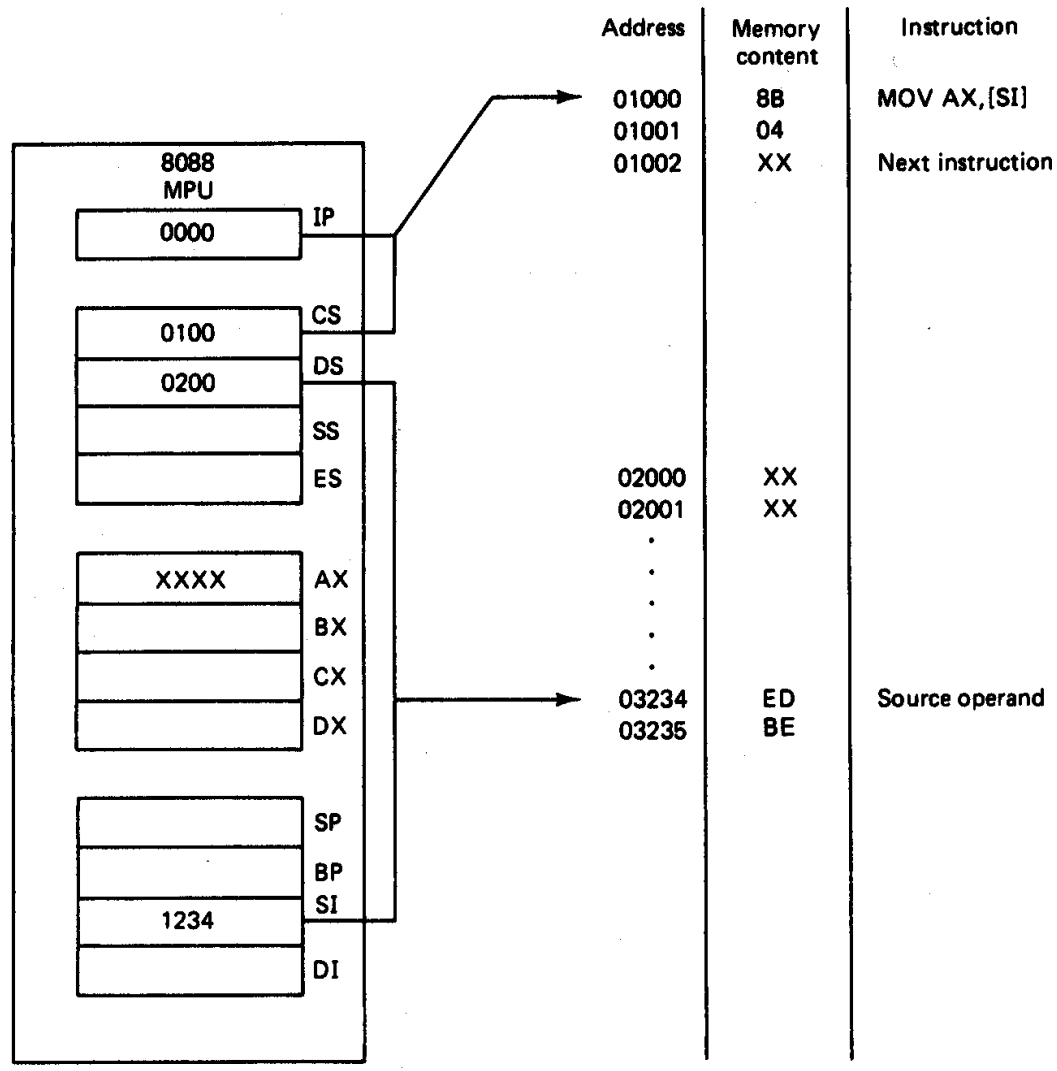
Indirect Addressing Mode

MOV AX, [SI]

$048B_{16}$

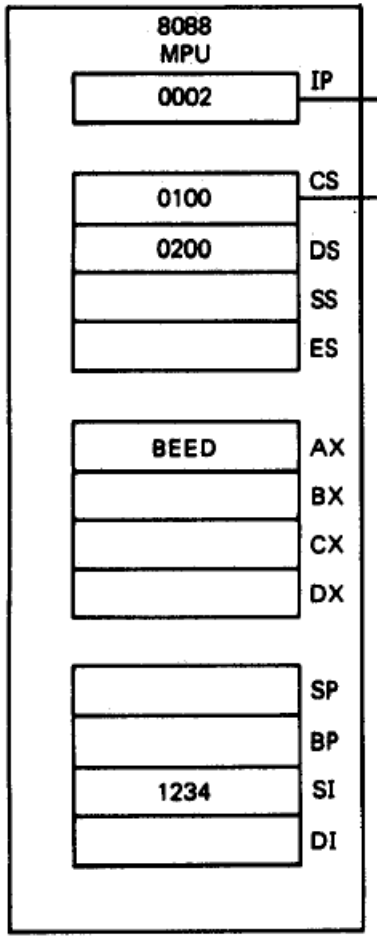
MOV AX, [SI]: This instruction moves the data at the memory location pointed to by the **Source Index (SI)** register into the `AX` register. Instead of a constant address, the memory location is provided indirectly through the register.

Before execution



(a)

After Execution



Address	Memory content	Instruction
01000	8B	MOV AX,[SI]
01001	04	
01002	XX	Next instruction
02000	XX	
02001	XX	
.		
.		
.		
.		
.		
03234	ED	
03235	BE	

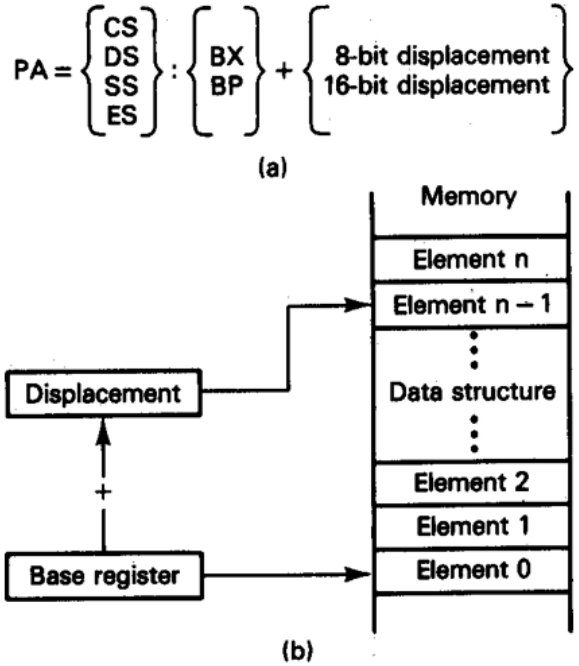
(b)

Based Addressing Mode

- MOV CX,[BX]+10 ⟹ PA = (DS)0 + (BX) + 10
- MOV AL,[BP]+5 ⟹ PA = (SS)0 + (BP) + 5

In this mode, the address is calculated using a base register and an offset. For example:

- **MOV CX, [BX]+10**: The operand is located at an address calculated by adding `10` to the contents of the `BX` register.
- **MOV AL, [BP]+5**: Here, the operand is fetched from the memory address given by adding `5` to the `BP` register.

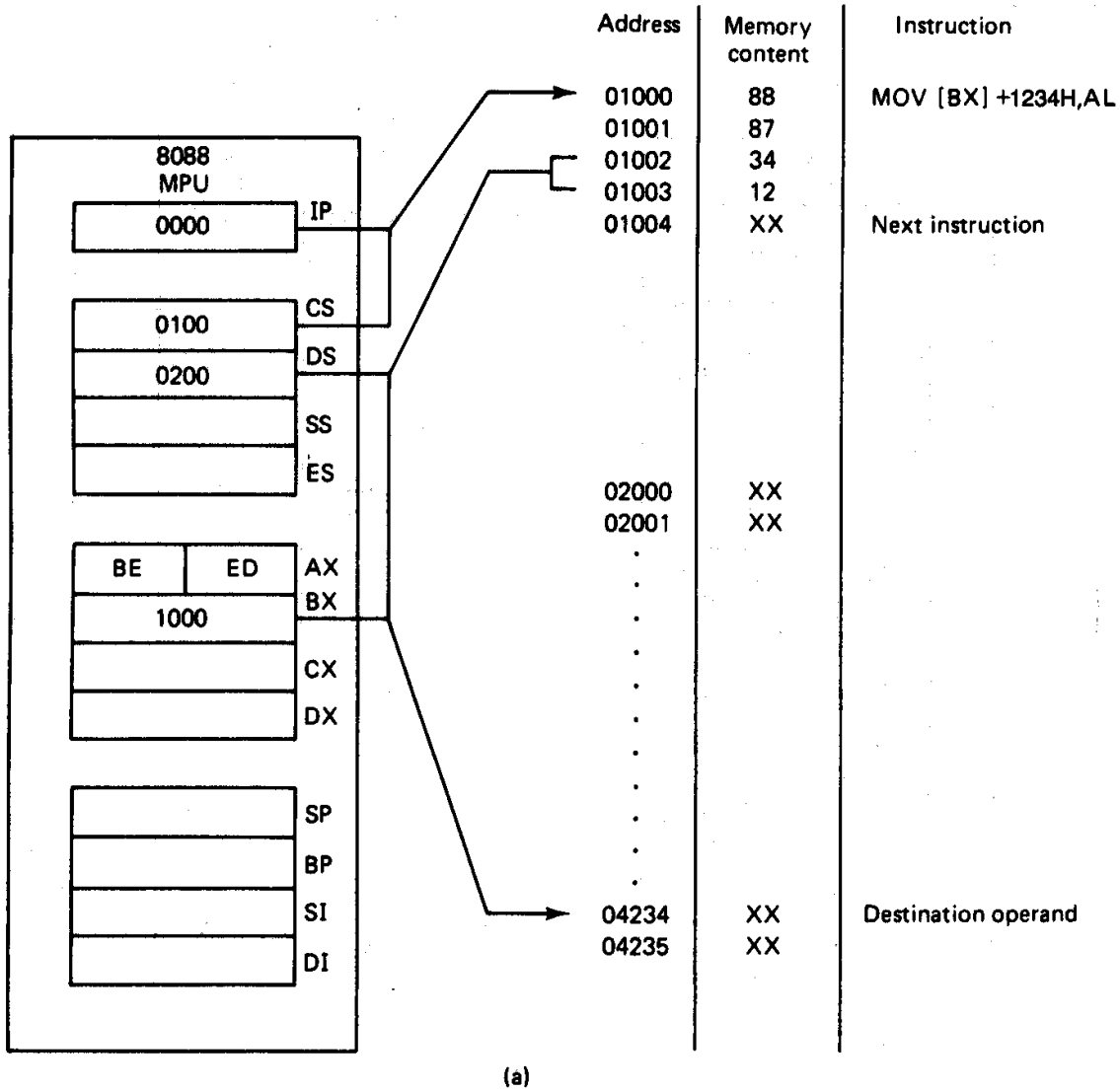


Example

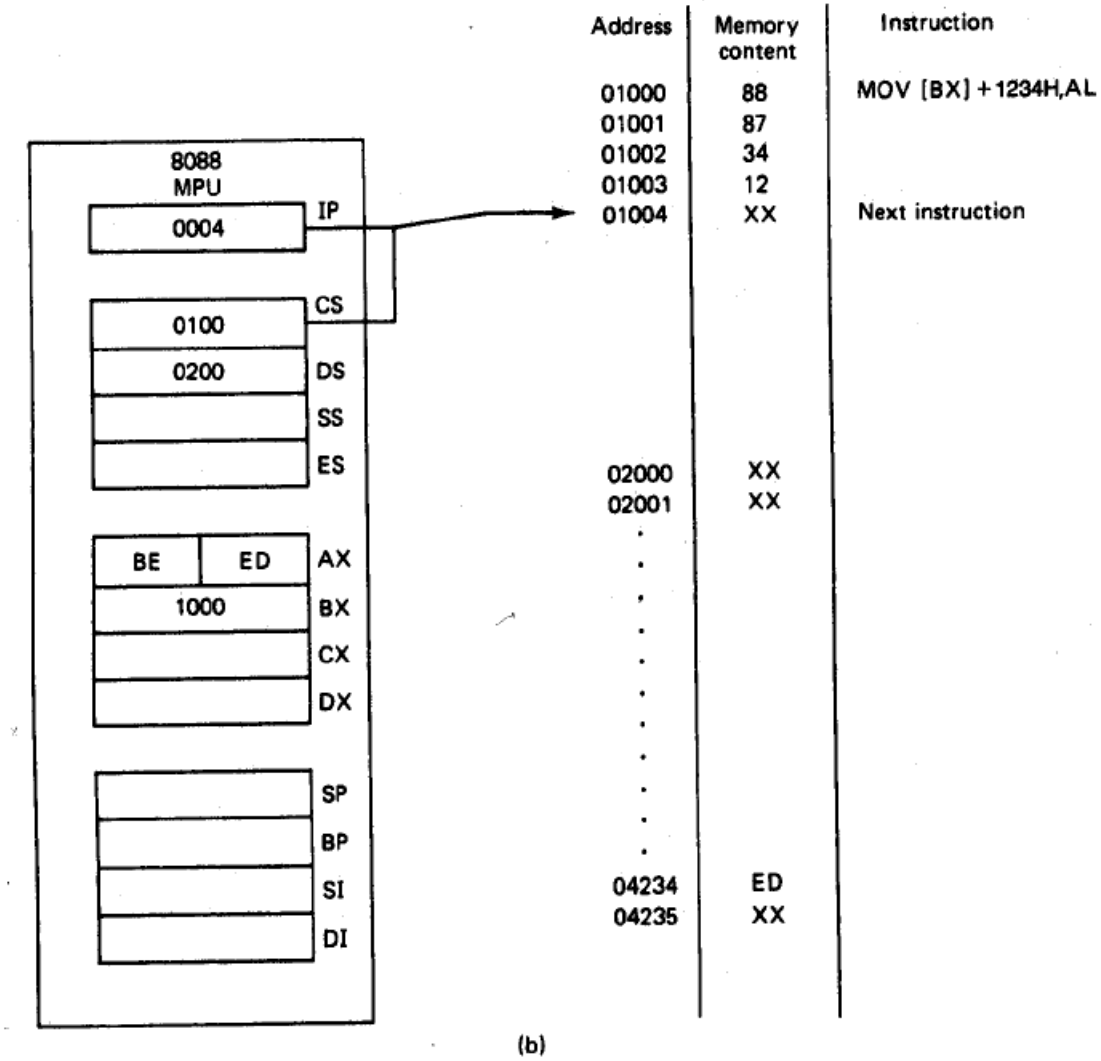
MOV [BX]+1234H, AL

12348788_{16}

Before execution



After Execution



(b)

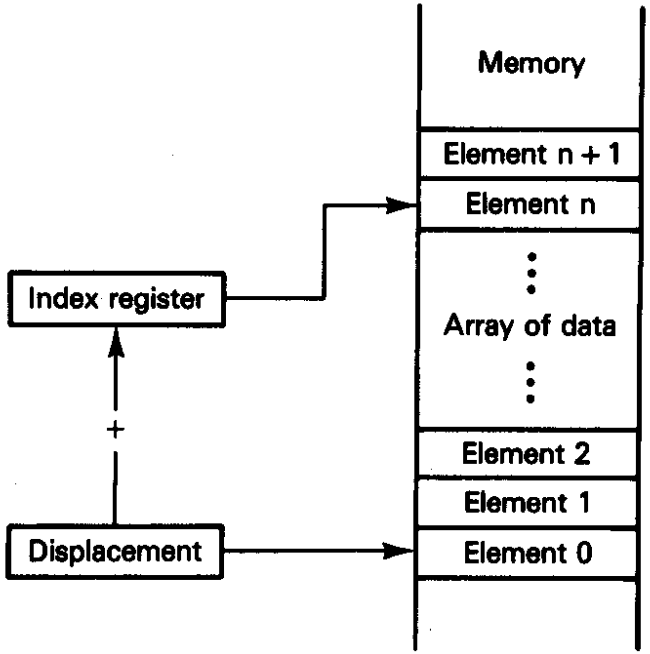
Indexed Addressing Mode

MOV D X, [SI]+5 → PA= (DS) 0 + (SI) + 5

MOV C L, [DI]+20 → PA= (DS) 0 + DI + 20

This mode involves an index register and an offset. For example:

- **MOV DX, [SI]+5**: This moves data from the address given by adding `5` to the `SI` register into the `DX` register.
- **MOV CL, [DI]+20**: This moves data from the memory address calculated as the `DI` register's value plus `20` into the `CL` register.



(a)

PA = Segment base: Index + Displacement

$$PA = \left\{\begin{matrix} CS \\ DS \\ SS \\ ES \end{matrix}\right\} : \left\{\begin{matrix} SI \\ DI \end{matrix}\right\} + \left\{\begin{matrix} \text{8-bit displacement} \\ \text{16-bit displacement} \end{matrix}\right\}$$

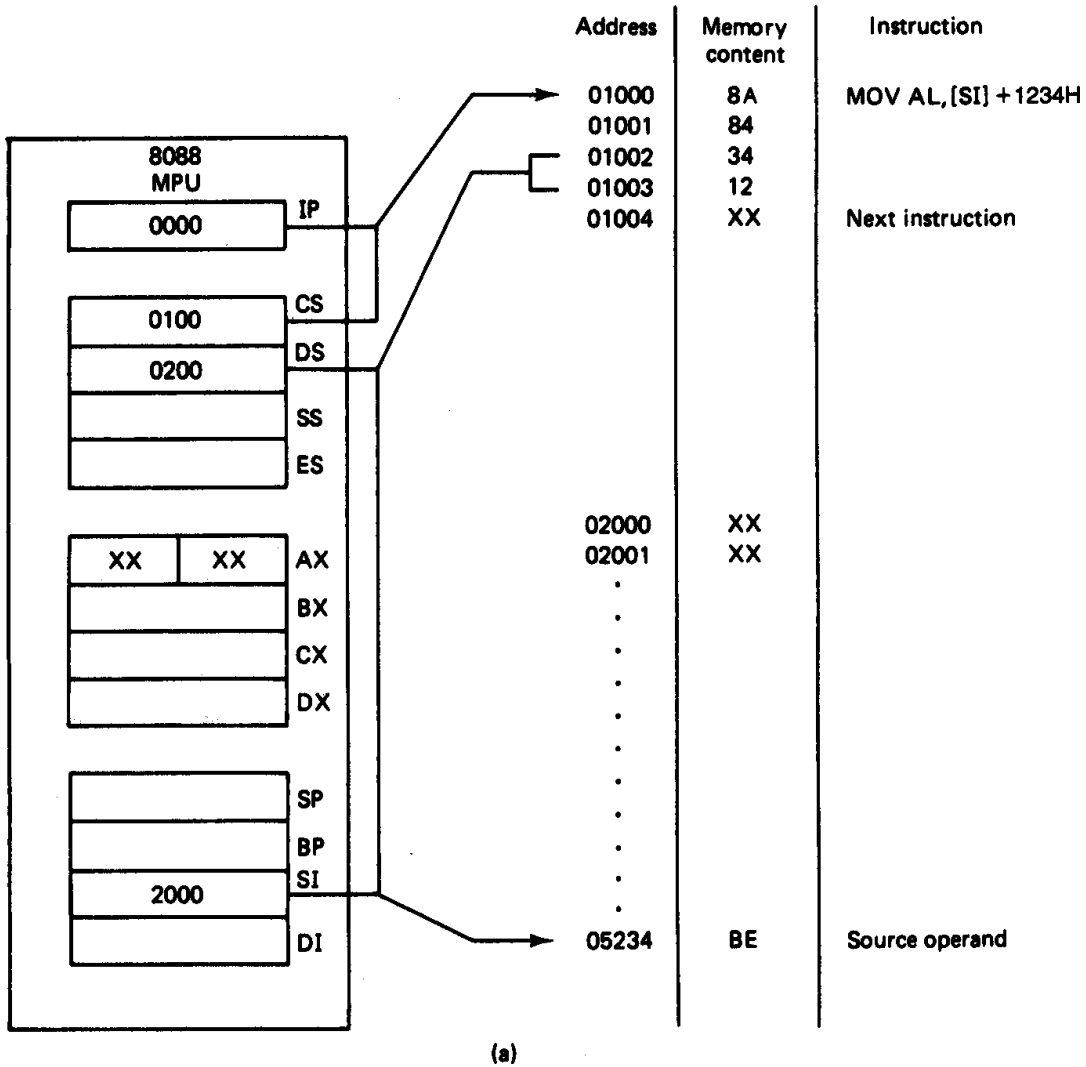
(b)

Example

MOV AL, [SI] +1234H

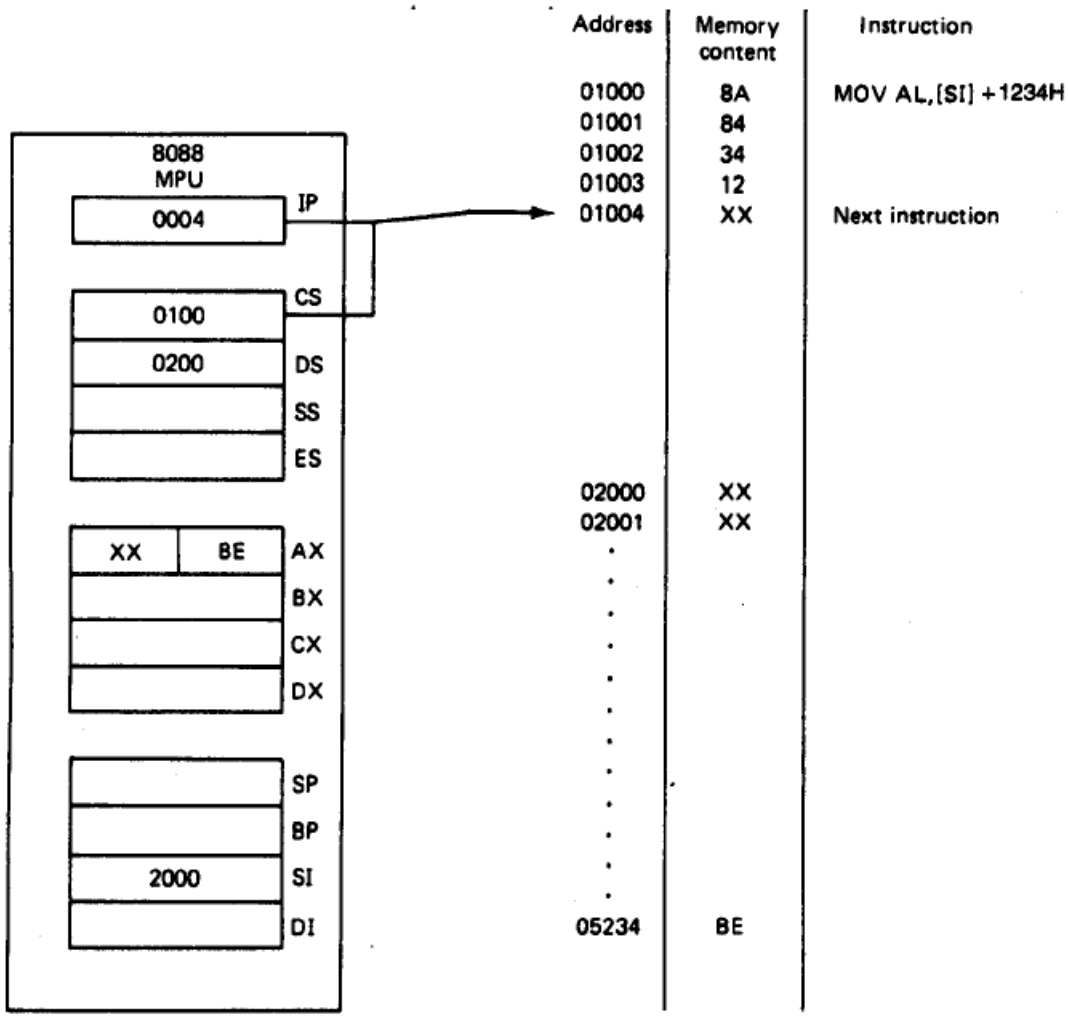
$1234848A_{16}$

Before execution



(a)

After Execution



(b)

Based Indexed Addressing Mode

- MOV CX, [BX][DI]+8h PA= (DS)0 + (BX) + (DI) + 8h

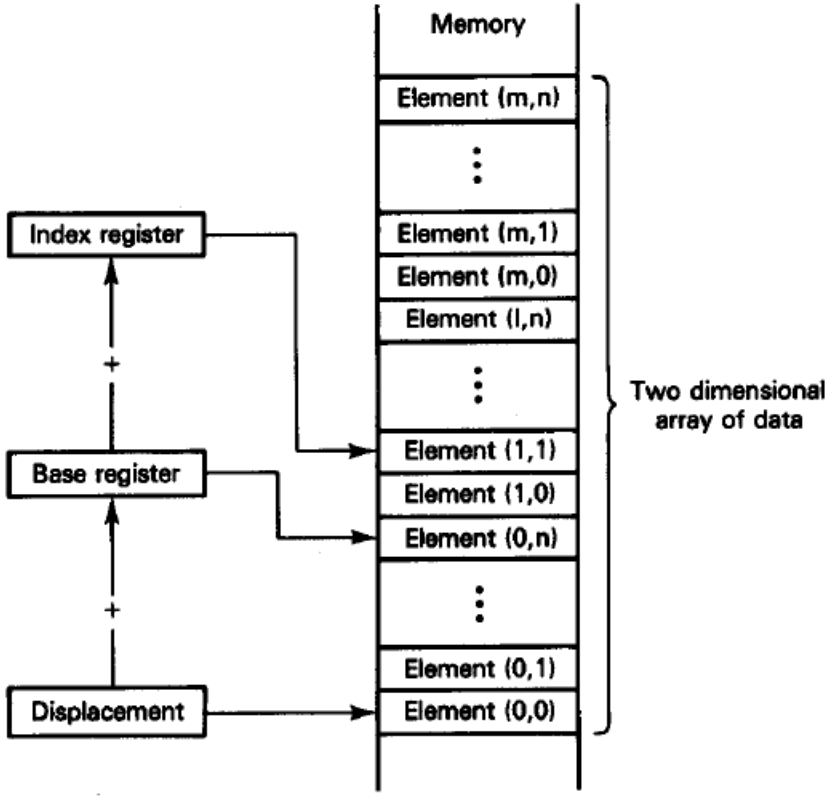
- MOV AH, [BP][SI]+29 PA: (SS)0 + (BP) + (SI) + 29

This combines both a base register and an index register, along with an optional displacement. For example:

- **MOV CX, [BX][DI]+8h**: The operand is accessed by adding `8h` to the sum of the values in the `BX` and `DI` registers.
- **MOV AH, [BP][SI]+29**: This instruction moves data from an address calculated by adding `29` to the sum of the `BP` and `SI` registers into the `AH` register.

- However
  MOV AX, [SI][DI] + displacement is illegal

Application



(a)

PA = Segment base: Base + Index + Displacement

$$PA = \begin{Bmatrix} CS \\ DS \\ SS \\ ES \end{Bmatrix} : \begin{Bmatrix} BX \\ BP \end{Bmatrix} + \begin{Bmatrix} SI \\ DI \end{Bmatrix} + \begin{Bmatrix} \text{8-bit displacement} \\ \text{16-bit displacement} \end{Bmatrix}$$

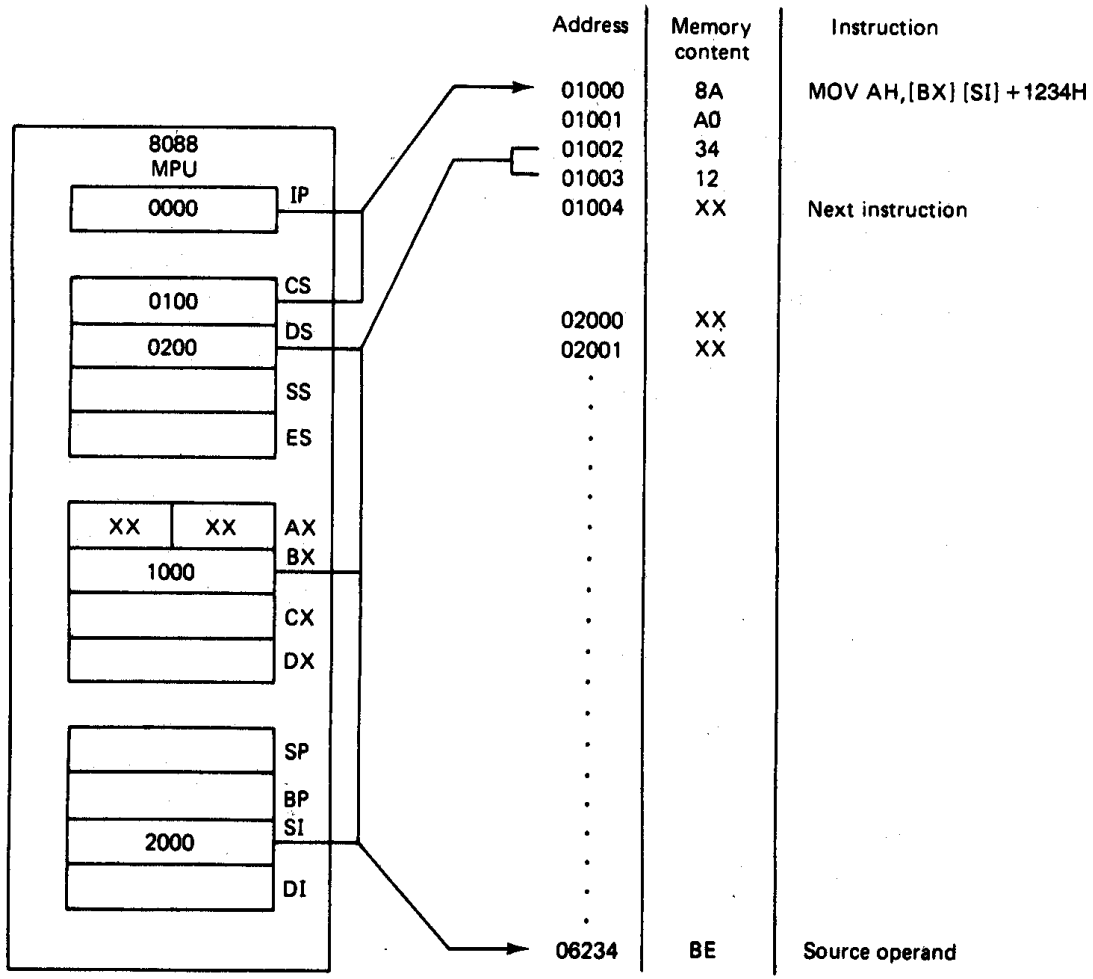
(b)

Example:

MOV AH, [BX][SI] +1234H

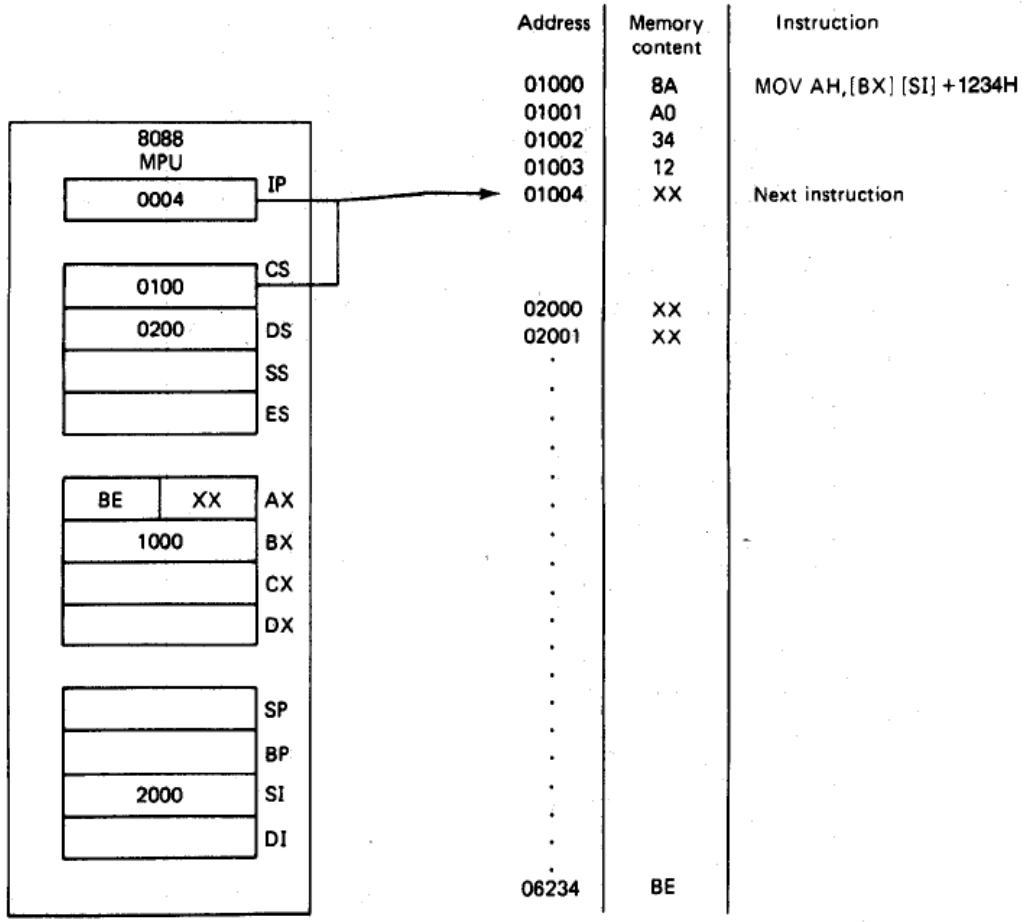
$1234A08A_{16}$

Before execution



(a)

After Execution



(b)

Note: Other variations of this mode, such as `MOV AH, [BP+SI+29]` or `MOV AH, [SI+BP+29]`, are possible. However, note that expressions like `MOV AX, [BX][BP] + displacement` are illegal.

Segment Override

Instruction	Segment Used	Default Segment
MOV AX,CS:[BP]	CS:BP	SS:BP
MOV DX,SS:[SI]	SS:SI	DS:SI
MOV AX,DS:[BP]	DS:BP	SS:BP
MOV CX,ES:[BX]+12	ES:BX+12	DS:[BX]+12
MOV SS:[BX][DI]+12,AX	SS:BX+DI+12	DS:BX+DI+12

Segment Override allows a program to use a segment register other than the default for a particular memory access. This is useful when accessing data across multiple segments, as it explicitly specifies which segment register (e.g., `ES`, `SS`) should be used for the operation.

Summary of Addressing Modes

The following addressing modes are commonly used in the 8086/8088 microprocessor:

- **Register Addressing Mode:** Operands are in registers.
- **Immediate Addressing Mode:** The operand is a constant.
- **Direct Addressing Mode:** The operand is located at a specified memory address.
- **Register Indirect Addressing Mode:** The memory address is given indirectly through a register.
- **Based Addressing Mode:** The address is calculated using a base register and an offset.
- **Indexed Addressing Mode:** The address is calculated using an index register and an offset.
- **Based-Indexed Addressing Mode:** The address is calculated using both a base and an index register, with an optional displacement.